

Lab 5 PID controller for robot navigation

In the previous Labs, you have been taught the following knowledge:

- how to use odometry data to control the robot moving from home to the charger position through two gaps.
- how to collect and store odometry and laser data for generating their data graphs.

It was an open control strategy as the robot is given several constant velocities and turning angles at several waypoints. *Note that you should keep a copy of [tutorial_pkg_node.cpp](#) written in the previous lab for backup in case you may need it, i.e.*

`cd ~/M-Drive/ros2_ws/tutorial_pkg/src`

`cp tutorial_pkg_node.cpp tutorial_pkg_node_lab 6.cpp`

In this lab, you should learn how to use PID controller to control your robot moving from home, go through two gaps and finally reach to the charger position, as shown in Figure 6.1. The velocity and turning angle of the robot will be adjusted by a PID control algorithm.

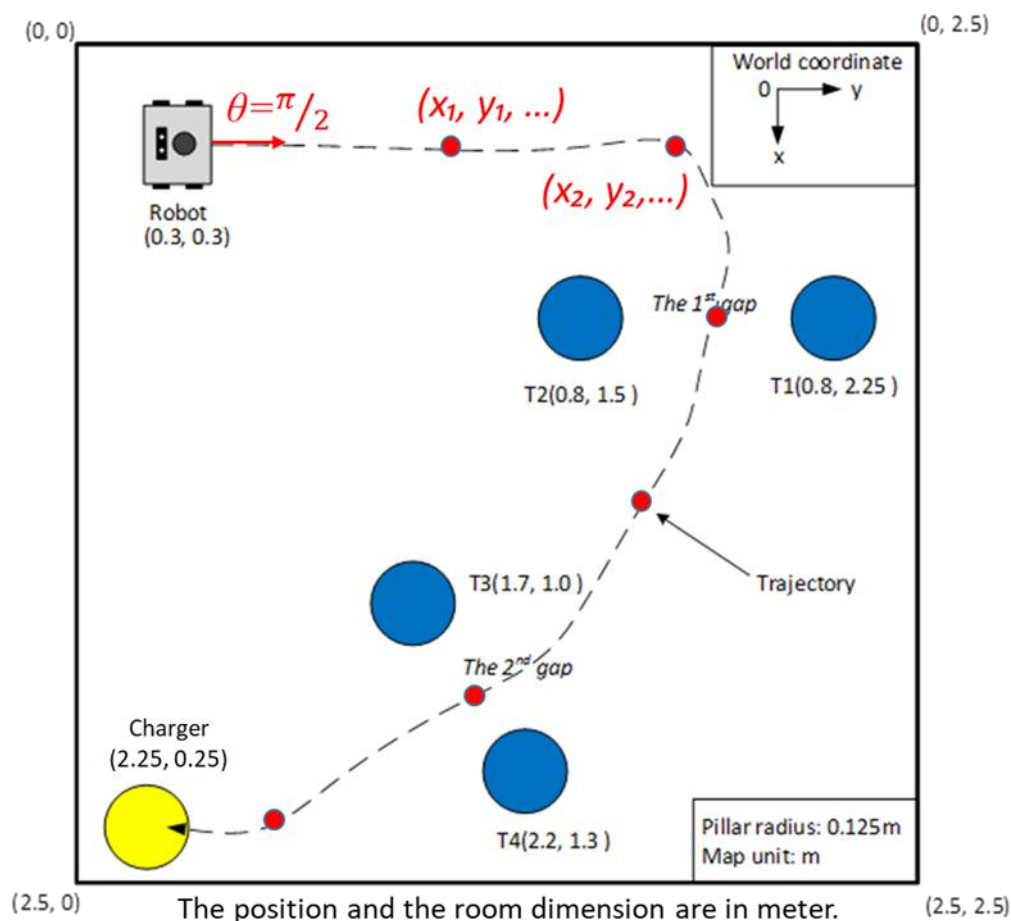


Figure 6.1 The navigation task

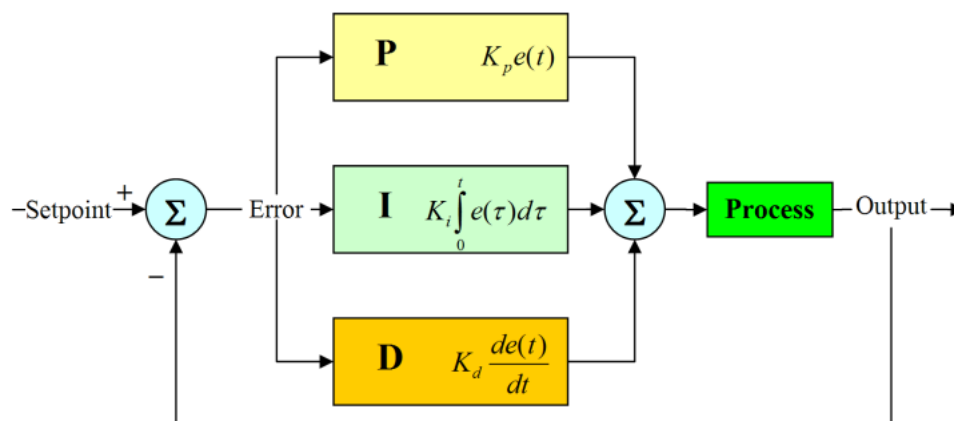
Task 1: Your task includes

- To write PID control algorithms for controlling the robot heading.

- To improve your PID control algorithms to drive the robot going through two gaps.
- To further improve your PID algorithms to drive the robot reaching the charger position.
- The parameters of the PID controller should be adjusted to make the trajectory smooth.
- The collected odometry and laser data should be used for PID based control, as well as saved into files.
- To generate four graphs, i.e., heading, trajectory, velocity, and the laser mapping. These graphs will be included into your Assignment report.

5.1 Basic Concepts

PID control is applied with remarkable success in various control application. All the consumer products have PID control. Some of examples include washing machines, air-conditioners, anti-braking systems used in vehicles, controllers used in traffic lights, large economic systems, etc. Figure 6.2 shows the configuration of a PID controller.



To create a PID algorithm using C/C++ language, you should follow select the initial values for its parameters which could be adjusted during the testing stage.

- Set the initial PID parameters: **kp** = 0.1, **ki** = 0.01, **kd** = 0.001
- Set the initial values: **ei_pre** = 0, **ed_pre** = 0.
- Calculate errors of the PID controller: **err**, **ei**, **ed**.

err = setValue – measuredValue

ei = **ei_pre** + **err**

ed = **err** – **ed_pre**

ei_pre = **ei**

ed_pre = **ed**

- Calculate the controller output: **Output** = **kp*** **e** + **ki*** **ei**+ **kd*** **ed**

Then, you should write C/C++ code to implement PID control algorithm for the robot navigation, which is shown in the next section.

5.2 PID controllers for the robot

Now, you should design several PID controllers for your robot to move from its home, go through two gaps and finally reach to the charger position, which are shown in YELLOW text below. Note that you may create different numbers of PID controllers to implement the specified navigation tasks if you prefer.

```
void Stopper:: scanCallback (const sensor_msgs::msg::LaserScan::SharedPtr scan)
{
    // ..... some existing code

    switch(stage){
    case 1: // wall following
        if (PositionY < odom_landmark1)
            // create "PID controller" for the robot to follow the wall
        else stage = 2;
        break;
    case 2: // the robot turns right into the 1st gap
        if (PositionX < odom_landmark2)
            // create "PID controller" for the robot to go to the middle of the 1st gap
        else stage = 3;
        break;
    case 3: // the robot moves toward the 2nd gap
        if (PositionX < odom_landmark3)
            // create "PID controller" for the robot to move towards the 2nd gap
        else stage = 4;
        break;
    case 4: // the robot going through the 2nd gap
        if (PositionX < odom_landmark4)
            // create "PID controller" for the robot to go to the middle of the 2nd gap
        else stage = 5;
        break;
    case 5: // the robot moves towards the charger position
        if (PositionY > odom_landmark5)
            // create "PID controller" for the robot to reach the charger position
        else stage = 6;
        break;
    case 6: // stop at the charger position
        moveStop();
        break;
    }
}
```

5.3 Create a PID controller for wall following

Step 1: Now you need to write C/C++ code to implement a PID controller. Using the following commands to open *tutorial_pkg_node.cpp*

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
```

```
gedit tutorial_pkg_node.cpp
```

You should add the following Yellow-highlighted PID code into your code.

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include <fstream>
#include <time.h>
#include <iomanip>

// .... existing code

struct Quaternion{double w, x, y, z;};
struct PID_para{double kp, ki, kd, ei_pre, ed_pre, Max_output;};
EulerAngles ToEulerAngles(Quaternion q){ //for calculating Th
// .... existing code

double PID_control(PID_para pid, double setPoint, double measuredData)
{
    double err = setPoint - measuredData;
    double ei = pid.ei_pre + err;
    double ed = err - pid.ed_pre;
    double output = pid.kp*err + pid.ki*ei + pid.kd*ed;
    if (output > pid.Max_outout )
        output = pid.Max_outout;
    else if(output < -pid.Max_outout)
        output = -pid.Max_outout;
    pid.ei_pre = ei;
    pid.ed_pre = ed;
    return output;
}
```

Step 2: To follow the wall, you need use Laser data, i.e., *leftRange*, to monitor how far the robot is from the wall. *leftRange* will be used as the input of the PID controller, and compared with the targeted range, i.e., *landmark1_toWall* = 0.3, to generate the turning control value so that the robot can follow the wall well. Note that the robot moving speed is constant in this case.

```
class Stopper : public rclcpp::Node{
public:
    // some existing code.....

    void PID_wallFollowing(double forwardSpeed, double laserData);
private:
    // some existing code.....
}
```

Step 3: Add the following subroutine before `void Stopper::scanCallback()`

```
void Stopper::PID_wallFollowing(double forwardSpeed, double laserData)
{
    PID para controller;
    double landmark1_toWall = 0.3; // the desired distance is 0.3m
    controller.kp = 0.5, controller.ki = 0.01;
    controller.kd = 0.01, controller.Max_output = 0.6;
    controller.ed_pre=0, controller.ei_pre=0;
    double PID_output = PID_control(controller, landmark1_toWall, laserData);
    moveForwardRight(forwardSpeed, PID_output);
}
```

Step 4: Add the following Yellow-highlighted code in `void Stopper::scanCallback()`. Note: You may need to remove the previous control function in `void Stopper::odomCallback()`, because you are using PID control.

```
void Stopper::scanCallback(const sensor_msgs::msg::LaserScan::SharedPtr scan)
{
    // ..... some existing code

    switch(stage){
        case 1: // wall following
            if (PositionY < odom_landmark1)
                PID_wallFollowing(FORWARD_SPEED_MIDDLE, leftRange);
            else stage = 6;
            break;
        case 2:

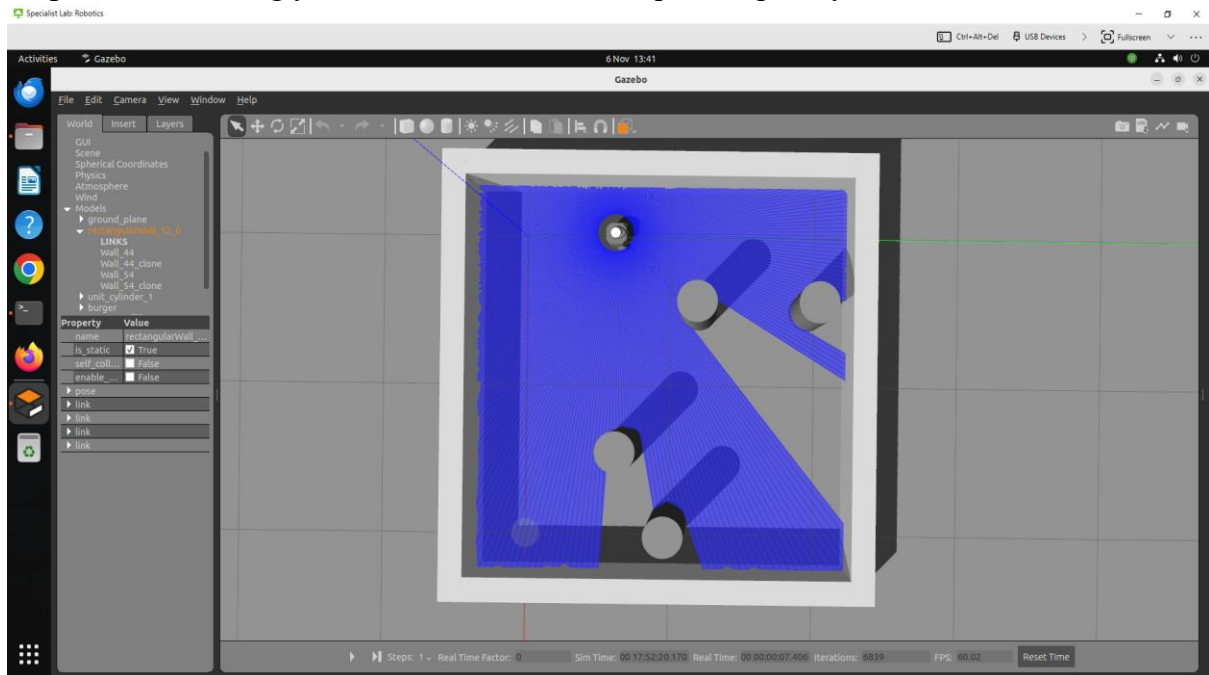
            // ,,,,,, some existing code

        case 6: // stop at the charger position
            moveStop();
            break;
    }
    // some existing code
}
```

Step 5: Run your code to evaluate your PID controller and adjust its parameters: firstly **kp**, then **ki**, to improve the wall following performance until the smooth trajectory is achieved.

```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
colcon build
ros2 launch tutorial_pkg tutorial_rosbot.launch.py
```

Step 6: After running your code, the robot will stop at the point you set, as shown below.



5.4 Create a PID controller for the robot to go to the middle of the 1st gap

Step 1: You should note that when the robot is in the middle of the first gap, its heading should be 0. Therefore, you need to set the targeted heading as *landmark2_heading* = 0. By using the odometry data, the PID controller will output the turning commands based on the difference between the measured heading and the target heading *landmark2_heading*. The following code is for your reference.

```
class Stopper : public rclcpp::Node{
public:
    // some existing code.....

    void PID_pass1stGap(double moveSpeed, double robotHeading);
private:
    // some existing code.....
}
```

Step 2: Add the following after *void Stopper::PID_wallFollowing()* method.

```
void Stopper::PID_pass1stGap(double moveSpeed, double robotHeading){
    PID_para controller;
    double robotHeadingGap1 = 0; // the desired robot heading should be 0 degree
    controller.kp = 2.0, controller.ki = 0.05;
    controller.kd = 0.01, controller.ei_pre = 0;
    controller.ed_pre = 0, controller.Max_output = 0.6;
    double PID_output = PID_control(controller, robotHeadingGap1, robotHeading);
    moveForwardRight(moveSpeed, PID_output);
}
```

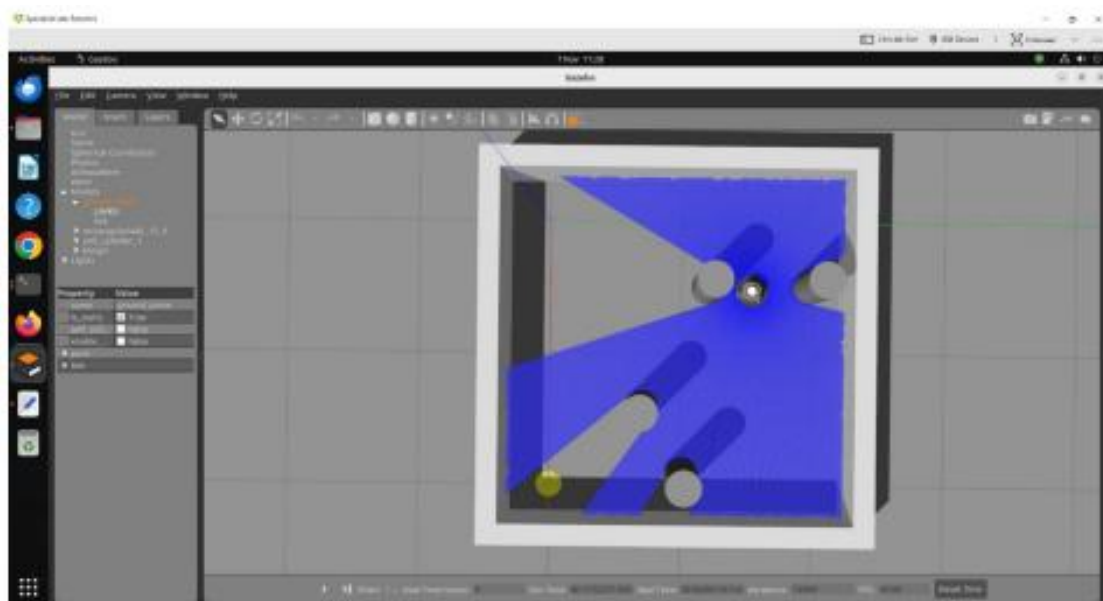
Step 3: Add the Yellow-highlighted command shown below.

```
void Stopper::scanCallback(const sensor_msgs::msg::LaserScan::SharedPtr scan) {
    // ..... some existing code
    switch(stage){
    case 1: // wall following
        if (PositionY < odom_landmark1)
            PID_wallFollowing(FORWARD_SPEED_MIDDLE, leftRange);
        else stage = 2;
        break;
    case 2: // going through the first gap
        if (PositionX < odom_landmark2)
            PID_pass1stGap(FORWARD_SPEED_MIDDLE, robotHeadAngle);
        else stage = 6;
        break;
    case 3:
        // .....some existing code
    case 6: // stop at the charger position
        moveStop();
        break;}
}
```

Step 4: Run your code to evaluate your PID controller and adjust its parameters: firstly **kp**, then **ki**, to improve the wall following performance until the smooth trajectory is achieved.

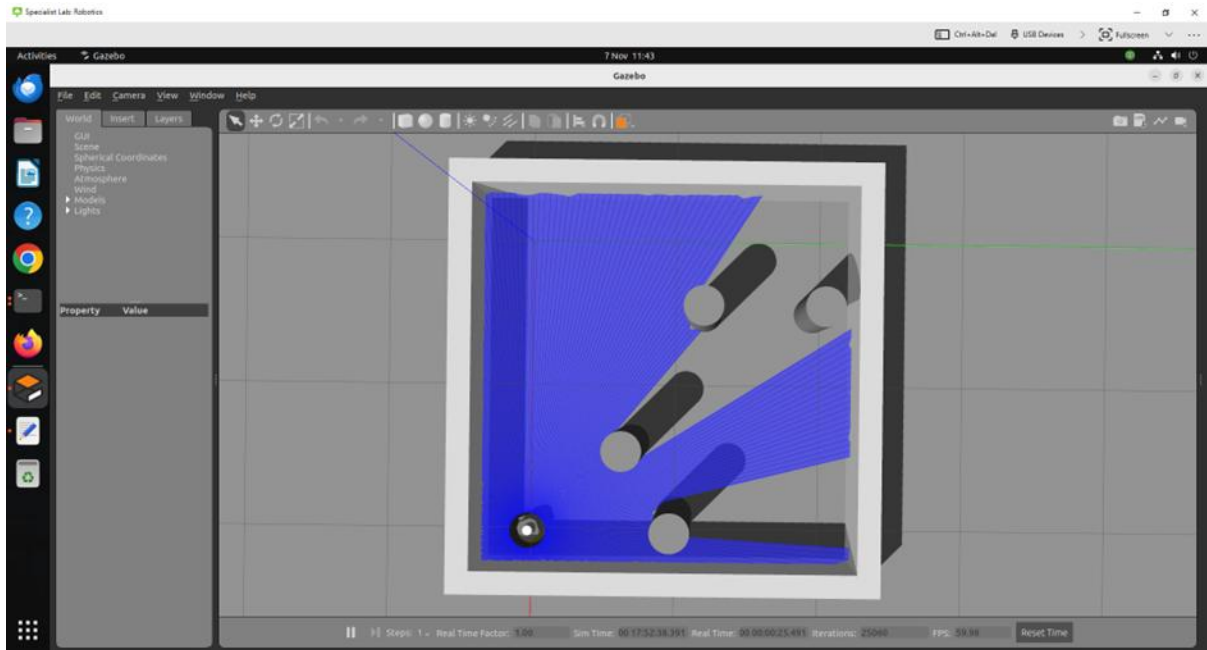
```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
colcon build
ros2 launch tutorial_pkg tutorial_rosbot.launch.py
```

After running your code, the robot should be stopped at the point you set, as shown below.



Step 5: Create remaining PID controllers

You should continuously create remaining PID controllers so that the robot can be controlled to move toward the 2nd gap, go through it, and finally reach the charger position.



```
switch(stage){
case 1: // wall following
    if (PositionY < odom_landmark1)
        PID_wallFollowing(FORWARD_SPEED_MIDDLE, leftRange);
    else stage = 2;
    break;
case 2: // going through the first gap
    if (PositionX < odom_landmark2)
        PID_pass1stGap(FORWARD_SPEED_MIDDLE, robotHeadAngle);
    else stage = 3;
    break;
case 3:
    if (PositionX < odom_landmark3)
        PID_moveto2ndGap(FORWARD_SPEED_MIDDLE, robotHeadAngle);
    else stage = 4;
    break;
case 4:
    if (PositionX < odom_landmark4)
        PID_pass2ndGap(FORWARD_SPEED_MIDDLE, robotHeadAngle);
    else stage = 5;
    break;
case 5:
    if (PositionX < odom_landmark5)
```



```
        PID_reachCharger(FORWARD_SPEED_MIDDLE, robotHeadAngle);  
    else stage = 6;  
    break;  
    case 6: // stop at the charger position  
    moveStop();  
    break;  
}
```

You should decide the reference values for the new three PID parameters, and adjust their parameters to improve the robot performance in an equivalent way conducted in the previous steps.

Note: After all robot behaviours are created, you should view saved odometry and laser data and create the velocity, trajectory, heading and map graphs for including into your Assignment report.